

# Machine Learning Pipelines

Module 29 Study Notes • Feature Engineering

## 1. The Problem: Manual Preprocessing

In previous modules, we handled data transformation step-by-step. While this is great for learning the mechanics, it creates massive problems in a real-world scenario. Imagine this workflow:

- Fill missing Age values using `SimpleImputer`.
- Encode categorical City values using `OneHotEncoder`.
- Scale the numerical features using `MinMaxScaler`.
- Train the `DecisionTreeClassifier`.

**The Production Nightmare:** When your model is deployed to a website and a user submits their details, you must manually write code to apply the exact same `SimpleImputer`, `OneHotEncoder`, and `MinMaxScaler` *in the exact same order* before predicting. If you change a single step during training, you must perfectly mirror it in production. This leads to brittle code and high chances of failure.

## 2. The Solution: Scikit-Learn Pipelines

A Pipeline chains multiple preprocessing steps together with a final machine learning model, creating a single, unified object. The output of step 1 automatically becomes the input for step 2.

**Crucial Benefit (Data Leakage Prevention):** A pipeline guarantees that transformations (like calculating the mean for imputation) are learned *only* from the training data. It automatically prevents test data from "leaking" into the calculation of those parameters, ensuring your evaluation is mathematically sound.

## 3. Building the Pipeline Architecture

To build a pipeline, you sequence your transformers (like `ColumnTransformer`) and your final Estimator. It requires a list of tuples containing a name and the object: `('name', object)`.

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
```

```

from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, MinMaxScaler
from sklearn.tree import DecisionTreeClassifier

# 1. Define ColumnTransformers for each preprocessing stage
# Note: Use column indices instead of names to avoid array conversion errors
trf1 = ColumnTransformer([
    ('impute_age', SimpleImputer(), [2]),
    ('impute_embarked', SimpleImputer(strategy='most_frequent'), [6])
], remainder='passthrough')

trf2 = ColumnTransformer([
    ('ohe', OneHotEncoder(sparse=False, handle_unknown='ignore'), [1, 6])
], remainder='passthrough')

trf3 = ColumnTransformer([
    ('scale', MinMaxScaler(), slice(0, 10))
])

# 2. Define Final Model
trf4 = DecisionTreeClassifier()

# 3. Bundle everything into the Pipeline
pipe = Pipeline([
    ('trf1', trf1),
    ('trf2', trf2),
    ('trf3', trf3),
    ('trf4', trf4)
])

```

## 4. Training and Deployment (The Magic)

Once the pipeline is defined, training all steps and the model becomes a single line of code:

```
pipe.fit(X_train, y_train)
```

To deploy the model, you export the **entire pipeline object** using the **pickle** library.

```

import pickle
pickle.dump(pipe, open('pipe.pkl', 'wb'))

```

## Production Code Simplification

In your website's backend, you no longer need to import the Imputer, Encoder, or Scaler. You just load the pipeline and pass the raw data.

```
loaded_pipe = pickle.load(open('pipe.pkl', 'rb'))

# The pipeline handles imputation, encoding, and scaling internally!
prediction = loaded_pipe.predict(user_input_data)
```

## 5. Hyperparameter Tuning with Pipelines

---

If you want to tune the parameters of the model *inside* the pipeline using **GridSearchCV**, you must prefix the parameter name with the name of the pipeline step followed by two underscores ( `__` ).

```
# Since we named our model step 'trf4' in the pipeline:
params = {
    'trf4__max_depth': [1, 2, 3, 4, 5]
}

grid = GridSearchCV(pipe, param_grid=params, cv=5)
grid.fit(X_train, y_train)
```